Report — PCM write-absorbing tier for an HBM + HBF KV cache

Last updated	**2026-10-22**
Simulator	`TokenSim/` (SimPy LLM-serving simulator with a Mooncake KV store)
Full record	`TokenSim/docs/findings.md` (append-only). If Part A and findings.md disagree, findings.md wins
Latest phase	**PHASE 14** (full-trace headline, HBF capacity check, break-even PCM endurance)
Tests	**277** passing
Paper	`../Latex/pcm-kv-micro.tex`, synced from Part A (see Pending LaTeX sync)

How this file is organised.

- **Part A** is the report: self-contained and current. It covers what was built, how it was measured, what was found, what was corrected, and what is not claimed.
- **Part B** is the working record: the weekly log, superseded numbers and the LaTeX sync list. It is kept unchanged for traceability.
- Anything marked **OPEN** is an unsourced parameter. Anything marked **ASSUMED** or **PLACEHOLDER** is used or recorded but not sourced.

Part A — the report

A1. The question

H3 (SK hynix, IEEE CAL 2026) adds high-bandwidth flash (HBF, NAND) behind HBM.

- H3 targets **read-only** data: model weights and shared, precomputed KV. For that data, NAND's low write endurance does not matter.
- **We put the same hardware under a write-heavy serving workload.** Here KV blocks evicted from HBM are spilled to HBF, so HBF receives a continuous write stream and its endurance is at risk.

We add a **bit-alterable non-volatile tier (PCM)** between HBM and HBF and ask **how much of the HBF write stream it can absorb, and at what cost in reuse.**

A2. The architecture being modelled

Per-GPU hardware

tier	capacity per GPU	source
HBM3e	192 GB, 8 cubes × 24 GB, all KV (weights live in HBF)	H3
HBF (NAND, TLC)	3 TB, 8 stacks × 384 GB, 3000 P/E	H3 (capacity); CLAUDE.md (P/E)
PCM (our addition)	**43.4 GiB**: 2 dies/stack × 35 mm² × 0.62 Gb/mm² = 43.4 Gb = 5.43 GiB per stack, × 8 stacks	0.62 Gb/mm²: 1st-gen 3D XPoint, Ehrmann *et al.*, *Applied Research* 2022

- **Topology.** HBM and HBF are daisy-chained through a router in the HBM base die; no HBM is given up for flash (H3).
- **PCM placement.** PCM sits **on top of each HBF stack, inside the existing footprint**, so it takes no NAND capacity.
- **Units.** Binary throughout: Gb = 2^{30} bits, GB read as GiB.

Data path

```
HBM (KV cache) --evict--> PCM (staging tier) --demote--> HBF (flash)
      ^                          |                            |
      +-------- read (hit) ----+----------------------------+
```

- **A block enters the store only when HBM evicts it** (spill).
- **On a later request, a block is read from PCM or HBF straight into HBM.** An HBF hit is **not** re-admitted to PCM: `store.get` reads it in place (`store.py:179-182`), and a later spill of the same key only refreshes it (`store.py:234-237`).
- **When PCM is full**, its LRU victim is either demoted to HBF or dropped, depending on the configuration.

The three configurations

	staging tier	admission write	at staging-tier eviction	what it represents
(b)	71,104-block **DRAM** buffer	**write-through**: the block also goes to HBF immediately	demote to HBF (`always`)	**the baseline**. Not "HBF only": the buffer serves 6.9% / 7.2% of (b)'s store-served tokens
(c)	71,104-block **PCM**	**write-back**: PCM only	demote to HBF (`always`)	removes the write-through copy

	staging tier	admission write	at staging-tier eviction	what it represents
(d)	71,104-block **PCM**	**write-back**	demote **only if the block was served from PCM at least once** (`if_read`), otherwise drop	**read-gated demotion**: our policy

Every number below is quoted as **Conv / T&A**, i.e. Mooncake Conversation trace / Tool&Agent trace.

A3. How it is simulated

Simulator

TokenSim is an event-driven (SimPy) LLM-serving simulator. Its **Mooncake store** has a memory tier and an offload tier, each with LRU eviction.

tier	role
HBM	the GPU prefix cache
Mooncake memory tier	the staging tier (PCM, or DRAM in (b))
Mooncake offload tier	HBF

- Each tier is a **per-operation latency function**, `latency = fixed_µs + bytes / bandwidth` (`mooncake/ssd.py`).
- Bandwidth is not shared between operations, and there is no queueing model for the far tier.
- **Write volume is counted exactly, in blocks and bytes, per tier.**

All PHASE 14 behaviour is added behind configuration flags whose default is the stock behaviour. With every flag at its default, output is byte-identical to the pre-PHASE-14 code (`ced3806`).

Model and block size

quantity	value	source
model	LLaMa2-70B-GQA: Nhead 64, KV heads 64/8 = **8**, Nlayer 80, Dmodel 8192, head dim 128	`hardware_models_h3b200.json:566-572`
precision	**FP16, hardcoded** (2 bytes per value)	`cache_config.py:46-52`
parallelism	TP2 / PP1 / DP4 on 8 GPUs. TP2 splits KV heads: **4 per shard**	`cache_config.py:173-178`
KV bytes per token, per TP shard	4 heads × 128 × 2 (K,V) × 2 B × 80 layers = **163,840 B**	`cache_config.py:46-52`
tokens per block	16	`--block_size 16`
bytes per block	16 × 163,840 = **2,621,440 B**	`mooncake_store.py:41-43`. Measured: bytes ÷ blocks = 2,621,440 in every run

Cross-check. 43.4 GiB per GPU ÷ 2,621,440 B = 17,776.64 blocks, floored to **17,776**. So the configured PCM is 17,776 × 2,621,440 B = 43.398 GiB per GPU, agreeing with the design to within the floor (+0.0036%).

FP16 is a limitation. FP8 would halve every byte count and double every tier's capacity in blocks.

Capacity bookkeeping: writing ranks and private pools

- **Only 4 of the 8 GPUs write to the store.** Only the TP-0 worker of each data-parallel group is scheduled (`llm_engine.py:577-579`). The TP-1 shard's stream is an exact mirror and is not simulated.

- **Every byte figure is therefore per TP shard** (one of 2). Ratios and reductions are unaffected.
- **Batch is quoted per active scheduler** = `batch_mean_cluster` ÷ 4.
- **Pools are sized for the 4 writing GPUs and split into 4 private partitions** (`store_partition = per_rank`). PCM is in-package on each GPU's own HBF stacks, so one GPU cannot evict another's blocks.

tier	pool knob	split in code	per writing GPU	recorded in every result
PCM	`memory_capacity_blocks = 71,104`	`71,104 // 4` (`store.py:446`)	**17,776 blocks = 43.4 GiB**	`partition_rank_memory_capacity_blocks = 17,776`
HBF	`ssd_capacity_blocks = 5,033,164`	`5,033,164 // 4` (`ssd.py:157-158`)	**1,258,291 blocks = 3 TiB** (3,298,534,359,040 B)	`partition_rank_ssd_capacity_blocks = 1,258,291`
HBM	from the hardware entry's `Capacity`	per GPU	**78,643 blocks = 192 GiB**	—

24 TiB, 12 TiB and 3 TiB are all correct; they measure different things.

- 24 TiB is the HBF of all 8 GPUs.
- 12 TiB is the HBF of the 4 writing GPUs, which receives the counted bytes.
- 3 TiB is one writing GPU's partition.

HBM `Capacity` is pre-compensated by +60.38 GiB. TokenSim subtracts the model weights from HBM, but H3 keeps the weights in HBF. This is a workaround, not a model change.

The headline configuration, and why it was chosen

```
memory_capacity_blocks = 71104     ssd_capacity_blocks   = 5033164
store_partition        = per_rank  store_partition_ranks = 4
hbm_evict_save_policy  = spill_only    # a block enters PCM only when HBM evicts it
store_lookup_start     = after_hbm     # look up only the blocks HBM did not hit
read_credit            = served        # if_read credits a block only when PCM actually served it
load_span              = missing       # load only the span HBM did not hit
cap 48 concurrent requests per scheduler; 30 QPS; full traces
```

Capacity and partition were chosen on hardware grounds (A2, above).

Entry, lookup and bookkeeping were chosen on modelling grounds. This is the only combination in which the store behaves as the intended hierarchy and PCM serves real reads. Each alternative fails in a specific, measured way:

alternative	what goes wrong	where
copy every prompt block into the store at admission (`prefill`, stock)	HBM and the store become parallel copies, not a hierarchy. PCM never serves a read (0 PCM-served tokens in 332 runs)	PHASE 12; PHASE 14 set A
spill-only, but the lookup starts at block 0 (`zero`, stock)	the head of the prompt stays in HBM, so the walk stops at the first absent key. The store serves 0 / 769 of 5,000 lookups	PHASE 14 STEP 3b
`read_credit = walked` (stock)	`if_read` credited every key the lookup *walked*, including blocks actually served from HBM. That opened the gate with no PCM read	PHASE 14 set A; V1
`load_span = from_start` (stock)	the load plan read the prompt head, not the span HBM missed (`mooncake_store.py:543`)	PHASE 14 set B

`after_hbm` follows the vLLM KV-connector contract: `get_num_new_matched_tokens(request, num_computed_tokens)` asks the external store only for tokens beyond the local hit. We cite the interface only.

Timing knobs (all runs)

knob	value	status
HBF read	0.1 µs, 7451 GiB/s (= 8 TB/s)	8 TB/s from H3. 0.1 µs is a **perfect-prefetch upper bound** for H3's latency-hiding buffer. This flatters (b)
HBF write	**200 µs, 3.0 GiB/s**	**TokenSim's stock SSD default, unsourced.** These are the values every run used
PCM read / write	0.1 / 1.0 µs, 100 / 20 GiB/s	**ASSUMED** (TokenSim defaults for the PCM media model)
`charge_eviction_writes`	true	demote writes are charged to the put that caused them

- **Byte counts are the results; timing only shapes the schedule.** PHASE 14 STEP 2 removed all store time, or scaled the PCM write timing 0.25× and 4×. Neither (b)'s HBF stream nor any PCM stream moved by more than 0.54%.
- That test ran under the earlier copy-at-admission model and has **not been repeated under the headline**.

Workload and replay

- **Traces.** Mooncake FAST'25 Conversation (**12,031 requests**) and Tool&Agent (**23,608**), replayed in full at **30 QPS** (`--trace_target_qps 30`).
- **They stand in for CAG** (cache-augmented generation), the H3 target workload. CAG's corpus size, sharing factor and transient KV are OPEN.
- **Prompt blocks only.** The store holds prompt KV. Generated KV has no key and never enters it. Generated tokens are **2.58% / 1.94%** of all tokens, which bounds what this omission can change on these traces.
- **Hit rates are floors.** The traces hash prompts in 512-token chunks, while the simulator uses 16-token blocks. A continuation therefore cannot reuse its predecessor's partial tail (PHASE 13).
- **Trace timestamps are whole seconds**, clustered into bursts. Order within a burst is file order.

Replicates

- **(b) and (c) are single runs.** Their HBF streams vary by ≤ 0.5% under perturbation, measured in STEP 2b and the STEP 3 prelude.
- **(d) is N = 4 per trace**: the unjittered run, plus 3 runs with **burst-level arrival jitter**, seeds 1–3.
  - Each burst is shifted by a random offset of up to j = 0.05 × the minimum inter-burst gap: **16.989 ms (Conv) and 33.338 ms (T&A)**.
  - The run fails unless arrival order still equals file order.
  - Every (d) figure is a range over the 4 runs, never a midpoint.
- **`--random_seed` is inert** on these traces: seeds 0/1/2 are byte-identical (PHASE 14 STEP 2b). That is why jitter is used to make replicates.

Metrics

metric	definition
HBF B/req	bytes written to HBF ÷ requests, **per TP shard**. The primary write metric; it is work-normalised
reduction (x)→(y)	1 − B/req(y) ÷ B/req(x)

metric	definition
residue-adjusted reduction	adds to HBF bytes the **end-of-run residue**: keyed blocks still in HBM at the end that were never spilled, assumed eventually demoted. Bounds the end-of-window bias
hit rate	prefix-cache hit rate (HBM + store)
no-store control / store benefit kept	the same run with the store disabled. Benefit kept = (hit − hit_no-store) ÷ (hit_(b) − hit_no-store)
PCM-served share	store-served tokens served from PCM ÷ all store-served tokens
HBF bytes per reuse-hit block	HBF bytes ÷ blocks hit on reuse (write cost per unit of reuse)
HBF read-back share	HBF-written blocks read back from HBF before the trace ends
PCM admissions by source	every PCM write tagged as first-time, after an `if_read` drop, or after an HBF eviction (`count_pcm_admit_source`)
`if_read` gate check	demotions whose block had ≥ 1 real PCM-served read. Must equal all (d) demotions
break-even PCM endurance	see A4, result 4

Verification gates (every campaign)

1. **Reproduction gate.** A new campaign must first reproduce the PHASE 9 cap-48 table to every digit (Part B).
2. **Byte identity.** With every new flag at its default, results are identical to `ced3806` after stripping wall time and pool-key samples.
3. **Counters are neutral.** Turning an instrumentation counter on changes no other key.
4. **Write identities.** In every (b)/(c) run, (b) HBF = admissions + demotions and (c) HBF = demotions, both exact.
5. **Pre-registered rules.** Expectations are written before the runs, and a failed rule is reported as failed.

A4. Results (full traces, headline configuration)

The claim has three parts, kept separate on purpose.

	part	nature
1	removing write-through halves HBF writes	**measured**, but an **identity**: a structural cost of the baseline
2	why the staging tier should be non-volatile and in-package	**argued, not measured**
3	read-gated demotion (d)	**measured, non-trivial**. The contribution

> **A same-capacity volatile buffer would write identical HBF bytes.** The case for PCM rests on durability and in-package density, not on write volume.

Result 1 — MEASURED, an identity: the baseline's structural write cost

We quantify a structural cost in the baseline and confirm that it does not depend on HBF capacity.

- **Write-through costs exactly one extra HBF write per admitted block.** In every (b)/(c) run, (b) HBF writes = PCM admissions + demotions, and (c) HBF writes = demotions.
- So removing the write-through copy halves HBF writes **whatever the medium**: `memory_media` changes timing only.

	Conv / T&A
(b)→(c), raw, 8 runs (finite and unlimited HBF)	**50.22–50.25%**
(b)→(c), residue-adjusted	**49.26–49.29%**
finite vs unlimited HBF capacity	**≤ 0.01 pt** (pre-registered rule: within 5 pt → "not driven by HBF capacity": **passes**)
HBF B/req (b) → (c)	3,036,078,553 → 1,511,160,257 / 1,466,116,938 → 729,379,925
hit rate (b) vs (c)	0.1898 vs 0.1898 / 0.4510 vs 0.4511 (keeps ~100% of the store's reuse benefit)
HBF evictions (finite HBF)	(b) 1.97 M / 1.60 M; (c) 1.90 M / 1.54 M. **Immaterial:** with unlimited HBF, HBF bytes move ≤ 0.09% and hit rate ≤ +0.11%. The evicted keys are cold
HBF occupancy with no capacity limit	peaks at 1.39× / 1.32× the pool. This is the trace's distinct-key footprint (an occupancy, not a capacity recommendation)

Unlimited HBF (`hbf_capacity_mode = unlimited`) is a sensitivity setting, not a hardware value.

Result 2 — ARGUED, NOT MEASURED: why the staging tier should be non-volatile and in-package

TokenSim does not support this part. It rests on write-back semantics and on capacity-per-area figures, not on byte counts.

Why not DRAM. The primary argument is durability.

- Under write-back, an acknowledged block lives only in the staging tier until it is demoted.
- A volatile buffer in that role would lose acknowledged data on power failure. This needs no density figure.
- TokenSim cannot measure durability.
- **Caveat:** KV blocks can be recomputed. For a cache, the loss costs recomputation (time and energy), not correctness. The argument is weaker here than for primary storage.

Supporting evidence only: placement.

- PCM's density is sourced: 0.62 Gb/mm², a planar die figure for 1st-gen 3D XPoint. That fits 43.4 GiB per GPU on top of the HBF stacks, **inside the existing stack footprint**, taking no NAND capacity.
- **No DRAM-density comparison is made** (decision 2026-10-22). Durability carries the "why not DRAM" argument, and the placement claim needs no DRAM figure. The newsletter placeholders stay in findings.md only.

Result 3 — MEASURED: read-gated demotion (d), the contribution

(d) is a **selection policy, not an identity**: it demotes only blocks that PCM actually served. N = 4 per trace.

	Conv / T&A
HBF writes vs (c)	(d) writes 0.97–1.10% / 1.28–1.34% of (c)'s HBF bytes, i.e. drops **98.7–99.0%**
HBF writes vs (b)	worst-case reduction **99.45% / 99.33%** (best 99.52% / 99.36%)
(d) HBF B/req	**14.6–16.6 M / 9.3–9.8 M**
(d) PCM B/req	**1.703–1.706 G / 0.815–0.816 G**. That is 11.6–11.7% / 10.6–10.7% more PCM writes than (c) (1.527 G / 0.737 G)
reuse kept vs (b)	(d) keeps **53.0–53.7% / 87.8–88.0%** of (b)'s reuse-hit blocks (hit rate 0.1006–0.1019 / 0.3959–0.3966 vs 0.1898 / 0.4510)

	Conv / T&A
…but most of that reuse needs no store	**No store at all keeps 48.3% / 85.8%** (hit rate 0.0916 / 0.3870). So (d) keeps **9.2–10.5% / 13.9–15.1% of the store's reuse benefit**. Over no store, its hit rate is +9.8 to +11.2% / +2.3 to +2.5%
PCM does the serving	PCM serves **72.6–76.5% / 71.3–73.9%** of (d)'s store-served tokens, against 6.3–7.2% in (b)/(c)
the gate is real	every (d) demotion had a real PCM-served read (difference 0, 8 runs)
where (d)'s PCM writes come from	first-time **89.54–89.67% / 90.23–90.29%**; **re-admission after an `if_read` drop 10.33–10.46% / 9.71–9.77%**; after an HBF eviction 0
why (d) writes more to PCM than (c)	those re-admissions account for **99.6–99.9% / 101.2–101.5%** of the (d) − (c) PCM-write difference, in blocks
HBF bytes per reuse-hit block	(b) 21.3 M / 6.06 M; (c) 10.6 M / 3.01 M; **(d) 193–217 k / 44–46 k**
lifetime ratio (b ÷ d)	×182–×208 / ×150–×157 (quoted only as a range; B/req is the robust figure)

Reading. (d) trades reuse for writes.

- It removes ~99% of the flash write stream.
- It keeps only a tenth to a seventh of what the store adds over no store at all.
- PCM carries the reuse it does keep.

It is the aggressive end of a write/reuse trade-off, not a free improvement.

Result 4 — break-even PCM endurance (a threshold, not a verdict)

Question. At what PCM endurance (cycles) do PCM and HBF exhaust their write budgets at the same moment?

```
E_PCM = (HBF_capacity × 3000 / HBF_B_per_req) × PCM_B_per_req / PCM_capacity
HBF_capacity = 1,258,291 × 2,621,440 B = 3,298,534,359,040 B   (per writing GPU)
PCM_capacity =    17,776 × 2,621,440 B =    46,598,717,440 B   (per writing GPU)
HBF endurance 3000 P/E (TLC)
```

- Both capacities must be on the same basis: both per writing GPU, or both pool. E_PCM is the same either way.
- Mixing them, e.g. a 12 TiB HBF pool against 43.4 GiB of PCM, is a 4× error.

	Conv	T&A
(b), the DRAM buffer read as if it were PCM	106,721	106,751
(c)	**214,535**	**214,657**
(d), range over N = 4	**21,726,973 – 24,762,024**	**17,725,533 – 18,584,178**
(d) ÷ (c)	101.3 – 115.4	82.6 – 86.6

Sanity check. E_d ÷ E_c = (HBF_c ÷ HBF_d) × (PCM_d ÷ PCM_c) holds in all 8 (d) runs. It is an algebraic identity, so it checks only the arithmetic.

Assumptions:

	assumption	status
1	perfect wear levelling in both tiers	**optimistic**. Relaxing it raises the (d) threshold
2	HBF write amplification = 1	**optimistic**. Relaxing it raises the (d) threshold
3	every write is a full-block write at both tiers	**optimistic**. Relaxing it raises the (d) threshold
4	each writing GPU's PCM:HBF write ratio equals the pooled ratio	**UNTESTED**. Per-rank write counts are not recorded

Further notes:

- Published chip-level PCM endurance already absorbs some cell variability. A comparison must not add a further levelling penalty on top of it; that would double-count.
- **End-of-run residue** (3.70–3.97% of would-be admissions on full traces):
  - if those blocks were spilled and demoted at each config's own rate, every threshold moves by −0.02% to −0.04%;
  - if they were spilled and never demoted, every threshold rises by +3.8% to +4.1% (upper bound).
- HBF also holds the model weights (60.38 GiB per GPU). Without static wear levelling, the HBF budget would be ~2% smaller.
- **No published PCM endurance figure is compared here.** That comparison (A4.1) is the author's to write.

Endurance as a swept axis. Because no endurance figure exists for the PCM generation whose density we use (A4.1), PCM endurance is treated as an axis, not a constant.

- Requests to first wear-out are computed for every assumed endurance from 10^5 to 10^{12} cycles. This is derived from the measured B/req; no new runs.
- A tier lasts capacity × endurance ÷ B/req requests, and the system lasts as long as its first tier.
- **(b)'s buffer is DRAM, so (b) is always HBF-limited.**

assumed PCM endurance	(d) ÷ (c), Conv	(d) ÷ (c), T&A	binding tier for (d)
10^5	0.89 – 0.90	0.90	PCM. (c) is PCM-limited too and lasts *less* than (b)
3×10^5	1.25	1.26	PCM
10^6	4.17 – 4.18	4.21	PCM
10^7	41.7 – 41.8	42.1	PCM
≥ 10^8	**90.8 – 103.3**	**74.7 – 78.3**	HBF (plateau = the HBF write ratio)

Two thresholds, with different meanings:

threshold (cycles)	Conv	T&A	meaning
(d) outlives (c)	**≈ 2.39–2.40 × 10^5**	**≈ 2.37–2.38 × 10^5**	where read-gating becomes **net-positive**. Below it, (d)'s extra PCM writes (+11.5–11.7% / +10.6–10.7% PCM B/req, almost all re-admissions) wear PCM out before (c)
(d) outlives (b)	≈ 1.19 × 10^5	≈ 1.18 × 10^5	where (d) beats the DRAM-buffer baseline
break-even E_PCM for (d)	2.17–2.48 × 10^7	1.77–1.86 × 10^7	above this, more PCM endurance buys nothing; (d) is HBF-limited

Script: `breakeven_pcm.py WORK sweep`. The figure for the paper is lifetime ratio vs assumed endurance (log x-axis) with the thresholds marked.

A4.1 Literature comparison — references verified

Supplied by the author on 2026-10-22. Every reference was checked against Crossref and/or the publisher on the same day (findings.md, "PCM ENDURANCE LITERATURE — verified references").

published figure	value	reference	use
spread of PCM endurance, mushroom-type cells	**10^6 – 10^9 cycles**	J. Zheng *et al.*, *Nature Communications* 16, 5788, 2025, doi:10.1038/s41467-025-60644-1 (open access)	the framing sentence for the spread

published figure	value	reference	use
record, ALD-confined cell with metallic liner	**2 × 10¹² cycles**	W. Kim *et al.*, "ALD-based confined PCM with a metallic liner toward unlimited endurance," *IEDM 2016*, pp. 4.2.1–4.2.4, doi:10.1109/IEDM.2016.7838343	cite this for the number, not a review
1st-gen XPoint module (Optane DC PMem)	**not published** ("N/A"; "we could not find any actual endurance data of the DCPMMs")	P. Götze, A. K. Tharanatha, K.-U. Sattler, arXiv:2001.02172, 2020, Table 1 (short paper at DaMoN 2020)	the negative claim only. The table compiles other sources, so it is not a source for its DRAM or NAND values
why figures spread (stuck-SET from segregation, stuck-RESET from voids; both atomic migration)	—	S. Kim, G. W. Burr, W. Kim, S.-W. Nam, *MRS Bulletin* 44(9), 710–714, 2019, doi: 10.1557/mrs.2019.205	framing
general PCM background	—	H.-S. P. Wong *et al.*, *Proc. IEEE* 98(12), 2201–2227, 2010; G. W. Burr *et al.*, *J. Vac. Sci. Technol. B* 28(2), 223–262, 2010	framing

Positions against our thresholds (stated as positions; the comparison is the author's to write):

figure	where it falls
mushroom-cell spread, 10⁶–10⁹	above (d)'s net-positive threshold (≈ 2.4 × 10⁵); overlaps (d)'s break-even (1.8–2.5 × 10⁷)
confined-cell record, 2 × 10¹²	above both

Generation mismatch. This must be stated in the thesis, not hidden.

- Our PCM **density** (0.62 Gb/mm²) is 1st-gen 3D XPoint.
- Intel never published write endurance for those products (Optane DC Persistent Memory).
- The circulating figures (50 k–500 k, 200 k, 10 M cycles) are analyst back-calculations from a petabytes-written rating. They are **not citable** and are used nowhere.
- So any published PCM endurance figure comes from a **different technology or generation** than the density figure.
- This is why endurance is a swept axis (Result 4), not a constant.

Measurement conditions (all configurations, full traces)

	Conv / T&A
end-of-run HBM residue	3.70–3.82% / 3.87–3.97% of would-be store admissions. It fell roughly in proportion to request count from W = 5,000, as pre-registered. It undercounts absolute B/req in every config
HBF read-back share	(b) 4.61% / 4.18%; (c) 9.24% / 8.56%; (d) 23.7–25.5% / 22.3–25.7%
HBF evictions	(b)/(c) only (Result 1). (d) never evicts. At W = 5,000 nothing evicts
batch per active scheduler	(b) 47.09 / 45.63; (c) +0.03% / −0.40% vs (b); (d) −0.59% to −0.66% / −1.86% to −2.06% vs (b). Arms are batch-matched

A5. Variants and the earlier reference window

W = 5,000 (the first 5,000 requests) was the headline window until 2026-10-12. Compare it with A4 by metric, not by digit: the replay pace and the jitter values differ. (d) here is pooled over N = 16.

	Conv / T&A
(b)→(c) raw	50.53% / 51.31% (residue-adjusted 48.38% / 46.49%)
(b)→(d) worst case	99.36% / 99.12%
(d) HBF B/req	14.8–20.7 M / 4.9–12.9 M
(d) hit rate vs (b)	−37.5% to −39.1% / −6.8% to −7.7%. Store benefit kept 15.4–18.8% / 9.8–20.6%
end-of-run residue	8.2–8.4% / 16.7–17.0%, the main reason the full traces became the headline

Variants, each measured under a different model and labelled with it:

variant	result	where
copy-at-admission (stock entry), (b)→(c)	50.81% / 50.99%	PHASE 14 headline set
copy-at-admission, (d)	**equivalent to no store**: with honest bookkeeping, 0 HBF writes and 0 store-served tokens	PHASE 14 V1
spill + from-zero lookup	degenerate: the store serves 0 / 769 of 5,000 lookups	STEP 3b
86.8 GiB/GPU PCM (142,213 blocks, global LRU) ≈ 2nd-gen XPoint density	(b)→(c) 51.10% / 52.18%	PHASE 9 STEP 3
(b)/(c) HBF wear vs concurrency (copy-at-admission)	0.50% / 1.10% spread over a 12× batch range. **Not established under the headline**, where HBM eviction is the store's only writer	PHASE 9 STEP 4
hit rate vs PCM capacity (copy-at-admission, global)	flat over a 150× capacity sweep	PHASE 10
falsifier: does HBM eviction drive HBF writes under copy-at-admission?	no: +0.00% / −0.07% over an 11× batch range	PHASE 12
shared (`global`) vs private (`per_rank`) pool at 71,104 (copy-at-admission)	(b)/(c) ≤ 0.22%. The (d) difference measured then (−13.8% / −24.5%) came from the lookup-bookkeeping artefact (A6 #3) and is not quoted	STEP 3a

A6. Corrections and retractions

These were found during the work. Each is in findings.md with its evidence.

#	what was reported	what is true	where
1	**PHASE 9 headline**: the cap-256 table, e.g. (b)→(c) 51.2% / 52.6%	cap 256 is a saturated operating point with large batch gaps ((d) vs (b) −4.7% / −7.7%). It was replaced by batch-matched cap 48, and later by the full-trace headline	PHASE 14 STEP 1
2	**The LaTeX headline did not match its own table.** Prose said ×90.3 / ×86.4 (PHASE 8, batch-unmatched); the table gave ×79.8 / ×58.1	ratios are quoted only as ranges paired with bytes/request	STEP 1e
3	**The lookup bookkeeping bug.** (d)'s 99.6%+ reductions under copy-at-admission came from `if_read` credit given to every key a lookup *walked*, including HBM-served blocks (`store.py:125`)	PCM never served a read under copy-at-admission (332 runs). With honest bookkeeping (d) ≡ no store. The read-back "mechanism" and every (d) variability analysis built on it (timing arms, replicates, concurrency sweep, pooled bound) are retracted or qualified as describing the artefact	set A; V1
4	**TP-rank-0 scheduling.** All 8 workers were assumed to write	only the 4 TP-0 workers are scheduled. The 142,213-block pool was 86.8 GiB per writing GPU (2× the design), batch figures were half the real value, and requests-to-wear-out against 24 TiB was 2× too high	STEP 3a

#	what was reported	what is true	where
5	**Shared global LRU.** One store LRU served all ranks	a block admitted by one GPU could evict another GPU's block (73.6–75.9% of evictions were cross-rank). Replaced by private per-rank pools, the faithful model of in-package PCM	STEP 3a
6	**`--random_seed` was taken as a replicate source**	it is inert on these traces. Replicates use burst-level arrival jitter	STEP 2b
7	the earlier jitter assumed 1 ms timestamp resolution	timestamps are whole seconds, so ±0.5 ms jitter only reorders requests within a burst (a workload variant). Replaced by burst-level jitter	STEP 2b
8	PCM tier configured at ~4 GiB (an early phase)	a 10× error. Every `if_read` result under it is **void**, not negative	CLAUDE.md, early phases
9	HBF-1/HBF-2 hardware entries (HBM given up for flash, −10% bandwidth derate)	wrong topology; superseded by `H3-B200-KVONLY`	CLAUDE.md
10	"HBF never evicts"	true only at W = 5,000. On full traces (b)/(c) evict 1.5–2.0 M cold blocks, which is immaterial (Result 1)	HBF capacity check
11	PHASE 12: "the previous output is a minority of what a continuation adds"	wrong for Conversation: output is 67.3% of the added tokens	PHASE 13
12	`--share_partial_tail` as a fix for the tail-reuse floor	it lowers the hit rate instead. The floor stands as a limitation	PHASE 13
13	"~10^8 PCM endurance" in early notes	unsourced. PCM endurance is OPEN, now framed as a threshold (Result 4)	STEP 1

A7. Open parameters

parameter	status	what was used
PCM endurance	**OPEN → a swept axis with thresholds** (Result 4). No published figure exists for the 1st-gen XPoint generation whose density we use (Götze *et al.*). Published PCM spread 10^6–10^9 (mushroom), record 2×10^{12} (confined): A4.1	none assumed; swept 10^5–10^{12}
HBF dies per stack	**OPEN**. Not needed for capacity (384 GB/stack is sourced)	—
TLC density 29 Gb/mm²	**OPEN** (uncited; unit unknown)	only to size the PCM die area
PCM die area 35 mm²	**OPEN**, derived from the uncited 29 Gb/mm² (1 Tb TLC die ≈ 35.3 mm², kept as 35)	2 × 35 mm² per stack
HBF write timing	**unsourced**: TokenSim's stock SSD default	**200 µs, 3.0 GiB/s** in every run
PCM media timing	**ASSUMED**	0.1 / 1.0 µs, 100 / 20 GiB/s
DRAM (HBM3/HBM3E) density	**not needed: dropped** (decision 2026-10-22). Placeholders stay in findings.md; not citable	not used anywhere
CAG corpus size, sharing factor, transient KV per request	**OPEN**	Mooncake traces stand in

A8. What is not claimed, and why

- **No latency or throughput numbers.**
  - This phase tests the write-absorption *idea*: how many bytes reach each tier, and what reuse survives.

  - TokenSim's store is a per-operation latency function with assumed or unsourced timing, no shared bandwidth and no far-tier queueing. Its timing is good enough to schedule the replay; it does not give latency or throughput results.
  - The arms are batch-matched at 30 QPS (cap 48 per scheduler, running at 45.6–47.1). Their timing differences are not results.
  - **Latency and throughput are the next phase**, on a simulator that models them.
- **No lifetime in days.**
  - Days need a write rate, and that needs a shared far-tier bandwidth model, which does not exist yet (STEP 6, planned).
  - Wear is given as bytes per request, as reductions, and as the break-even threshold.
- **No verdict on PCM wear-out.** Result 4 is a threshold, not a comparison with published figures.
- **The density and endurance figures come from different generations.** No endurance is published for the 1st-gen XPoint products whose density we use (A4.1).
- **No claim that PCM's medium causes the 50%.** It is a write-policy identity (Result 1).
- **No concurrency-invariance claim under the headline.** It was measured only under copy-at-admission (A5).
- **Generated KV is out of scope.** The store holds prompt blocks only (2.58% / 1.94% of tokens are generated).
- **FP8 is not modelled.** Precision is hardcoded FP16; FP8 would halve all bytes.
- **No NAND-level write amplification.** WAF = 1 is assumed, and that is optimistic. A NAND-level simulation (MQSim) is out of scope for this work.

A9. Next steps

	item	status
1	Literature for A4.1 is verified. The author is downloading the paywalled papers (`Documents/Research_papers/README.md`)	in hand
2	LaTeX micro-paper rewrite from Part A	next
3	STEP 5: `--content_aligned_tail` (tail-sharing sensitivity)	not started
4	STEP 6: shared far-tier bandwidth model (`offload_bandwidth_model = per_op | shared`), **ASSUMED** 30 / 48 GB/s per stack (27.94 / 44.70 GiB/s). Mechanism only, no experiments	not started
5	latency/throughput on a timing-faithful simulator	next phase
6	per-rank write counters (to test assumption 4 of Result 4)	not built
7	Option 2: store generated KV (`TokenSim/docs/option2-output-kv-scope.md`)	scoped, not built
8	MQSim WAF/GC run to replace WAF = 1	**out of scope** (decision 2026-10-22). WAF = 1 stays an optimistic assumption
9	CAG workload definition	OPEN

Part B — working record

> Kept for traceability. Numbers here may be superseded by Part A. The earlier "Status at a glance" and "Next steps" sections are replaced by Part A; their history is in the weekly log below and in findings.md.

Weekly log

> Newest first. Keep entries short: what was done, what moved, what is next.

2026-10-17 to 2026-10-22 — PHASE 14: framing, HBF capacity settled, break-even endurance, report rewrite

- **Framing decided.** (c)'s halving is a write-policy identity that characterises the baseline; (d) is the contribution. The case for a non-volatile in-package tier is argued, not measured, and durability leads the "why not DRAM" argument.
- **HBF capacity settled.** 5,033,164 is the pool; the code splits it into 1,258,291 blocks (3 TiB) per writing GPU (`ssd.py:157-158`, recorded as `partition_rank_ssd_capacity_blocks`). Block size was confirmed two ways at 2,621,440 B.
- **Break-even PCM endurance:**
  - (c) ≈ 214.5 k cycles;
  - (d) 21.7–24.8 M (Conv) / 17.7–18.6 M (T&A);
  - three optimistic assumptions and one untested.
- **(d) PCM admissions by source, N = 4.** Re-admission after an `if_read` drop is 10.33–10.46% / 9.71–9.77% of (d)'s PCM writes. That accounts for essentially all of (d)'s extra PCM writes over (c).
- **DRAM density recorded as a PLACEHOLDER** (newsletters; needs a primary source).
- **report.md restructured.** Part A is a self-contained report of what was built, how it was measured, results, corrections, open parameters and what is not claimed. Part B keeps the working record.
- No simulator code changed in this period.

2026-10-12 — PHASE 14: HBF capacity check, admission counter, full-trace headline

- **Decisions:**
  - **Full traces are the headline window.** W = 5,000 is the earlier reference.
  - (b) is described exactly: a 71,104-block write-through DRAM buffer in front of HBF.
  - CLAUDE.md now says HBF eviction depends on the window.
- **New flags:**
  - `hbf_capacity_mode` (`finite` | `unlimited`): a sensitivity setting, not a hardware value.
  - `count_pcm_admit_source`: tags each PCM write as first, after an `if_read` drop, or after an HBF eviction.
  - Both are stock by default. 277 tests pass. Byte identity against `ced3806` holds, and the counter reproduces STEP 4 exactly.

- **Rule passes.** (b)→(c) changes by ≤ 0.01 pt with unlimited HBF, so **(c)'s halving is not driven by HBF capacity**. It is an identity: (b) HBF = admissions + demotions, and (c) HBF = demotions. That makes it a write-policy effect.
- **HBF evictions are of cold keys.** They cause 0.02–0.09% of PCM admissions and move HBF bytes by ≤ 0.09%. This corrects the STEP 4 note that called them material re-writes.
- **(d) re-admissions after an `if_read` drop:** 10.46% / 9.77% of (d)'s PCM writes.

2026-10-12 — PHASE 14 STEP 4 (full traces, headline configuration)

- **14 runs.** Calibration on T&A (b): 8:42 and 11.13 GiB, within the estimate. The rest ran 2 at a time; 54 min in total.
- **(c) ≈ 49.3–50.3%** on full traces: raw 50.23 / 50.25%, residue-adjusted 49.29 / 49.26%. The raw-adjusted gap shrank from 2.2 / 4.8 points to 0.9 / 1.0. Store benefit kept ~100%.
- **(d):** worst case 99.45 / 99.33%. Its hit-rate cost grows on full traces (−46 to −47% / −12%), and it keeps only 9–15% of the store's reuse benefit.
- **Residue share** 3.7–3.8% / 3.9–4.0%: falls roughly in proportion to request count, as pre-registered.
- **New regime:** HBF fills in (b)/(c) on full traces (1.5–2.0 M evictions); (d) none.
- **Code reading:** the HBF → HBM read path bypasses PCM. There is no re-admission of HBF-served blocks.
- **Quoting notes recorded:**
  - (c)'s T&A benefit kept is quoted as ~100%, not 104.3%.
  - (c) was ~48–51% until this step; on full traces it is ~49.3–50.3%.

2026-10-12 — PHASE 14: bookkeeping fixes verified (V1, V2)

- **Correction notes:**
  - The read-back mechanism is **retracted** as a mechanism.
  - All (d) HBF variability analyses are **qualified** as characterising the artefact.
  - **(c)** is the claim that holds in every model: 50.81 / 50.99%, and 50.53 / 51.31% under spill.
- **New flags** `read_credit` (`walked` | `served`), `load_span` (`from_start` | `missing`) and the `count_if_read_gate` check. Byte-identical at default; tests **263**.
- **V1:** (d) HBF = 0, 0 gate passes. The artefact diagnosis is confirmed.
- **V2:** gate passes backed by PCM-served reads, difference 0. (b)→(c) 50.53 / 51.31%; (b)→(d) worst 99.36 / 99.12%. (d) HBF 14.8–20.7 M / 4.9–12.9 M. (d) hit rate vs (b) −37.5 to −39.1% / −6.8 to −7.7%.
- **No headline change**; decision pending.

2026-10-12 — PHASE 14: vLLM-style lookup, sets A and B

- **New `store_lookup_start`** (`zero` stock | `after_hbm`, the vLLM connector contract). Byte-identical at default; tests **255**.
- **Set A (headline + after_hbm) fails rule A2.**
  - (b) and (c) are identical to the headline.
  - (d) writes **0** to HBF in every run, and its store lookups never hit.
  - The cause is a **correction**: under copy-at-admission PCM never serves a read (all 332 runs). The headline (d)'s gate was opened by lookup touches of HBM-resident heads (`store.py:125`), so its HBF writes are an accounting artefact.
  - Headline tables not rewritten; decision pending.

- **Set B (spill + after_hbm)** is a working spill hierarchy: PCM serves real reads, and (b)/(c) keep the headline hit rate.
  - (d) HBF is 18.8–24.4 M / 5.3–15.3 M, above the bands. (b)→(d) ≥ 98.96%.
  - Rules: (i) passes; (ii) and (iii) fail.
  - The headline numbers depend on the PCM entry model.
- **Recorded:** the from-zero spill result ("copy-at-admission required under this lookup"), and a stock load-plan constraint: the head keys are read, not the external span (`mooncake_store.py:543`).

2026-10-12 — PHASE 14 STEP 3b (robustness: HBM spill)

- **New `hbm_evict_save_policy = spill_only`**, with end-of-run residue and admission-to-first-spill counters. Byte-identical at default; tests **250**.
- **Verdict (pre-registered rules): the headline does not hold under spill; the result depends on the PCM entry model.**
  - Rule (iii) fails on both traces: (d) HBF is 0 / 0.37 M B/req, below the bands.
  - Rule (ii) (residue-adjusted) passes on conv (≤ 4.89%) and fails on T&A ((b) HBF −6.21%).
  - Rule (i) passes.
- **Cause, measured:** under spill_only the store serves almost no reads. It hits 0 / 5,000 (conv) and 769 / 5,000 (T&A) lookups, against ~4,990 in the headline. The prompt head stays in HBM and is never spilled, and `store.lookup` stops at the first absent key. So `if_read` never passes.
- **Open:** a lookup that starts after the local GPU hit. Not built; needs a decision.
- **Future-work line added:** a stricter demotion gate than "read once in PCM".

2026-10-11 — PHASE 14 headline follow-ups

- **Bib entry** `ehrmann2022pcm` uses the supplied fields: Ehrmann, Blachowicz, Ehrmann, Grethe, "Recent developments in phase-change memory", *Applied Research* 1(4) e202200024, 2022.
- **Read-back share is a lower bound** (in-window only). The (b) figure, 3.90% / 2.98%, is an observation pending STEP 4.
- **De-confounded read-back test.** At global @ 71,104 vs global @ 142,213 (capacity only, 8 new runs), **98.2–98.4% (T&A) / 70.6–76.2% (Conv)** of (d)'s avoided HBF writes were not read back in-window. That agrees with the per_rank comparison, so the mechanism is **lifted from "candidate"**. It stands as a within-window result: those blocks pass `if_read` at the larger pool and then go unread from HBF.

2026-10-11 — PHASE 14 headline set (design capacity, per_rank)

- **Unit resolved: binary.** Pool 71,104 = 4 × 17,776 = 43.4 GiB/GPU. 0.62 Gb/mm² is now cited (Ehrmann *et al.* 2022) in the LaTeX.
- **New `count_hbf_readback` counter.** Byte-identical when off and behaviourally neutral when on. Tests **242**.
- **Headline set** (36 runs; (d) N = 16 per trace):
  - (b)→(c) **50.81% / 50.99%**.
  - (b)→(d) worst case **99.66% / 99.63%**.
  - (d) HBF B/req 5.95–11.70 M / 4.08–6.35 M.
  - (d) hit rate vs (b) −43.0% to −45.9% / −8.6% to −9.2%.

- **Read-back test of the candidate mechanism.** Of the HBF writes (d) avoids at the design pool vs the 86.8 GiB/GPU point, **97–98% (T&A) / 70–80% (Conv)** were blocks never read back from HBF. This is consistent with the candidate, but pool size and partition change together.
- **(b) and (c) read back only 3–4% / 7–8% of what they write to HBF, in-window.** That is a **lower bound**: reads after the W = 5,000 window are not counted, which biases it against late writes. It is an **observation pending STEP 4**, not a finding.
- **Under `per_rank`, (d) is stable to small jitter:** all 5 seeds at j = 0.005 reproduce the stock run on both traces.

2026-10-11 — PHASE 14, decision record (docs only)

- **Headline = design capacity + `per_rank`.** Decided on hardware grounds: die area fixes 43.4 GiB/GPU, and PCM is private per GPU. 142,213 is now the 86.8 GiB/GPU point. Recorded that this moves (c) down slightly and (d) up.
- **OPEN unit check:** binary 71,104 vs decimal 66,220. It also bears on the TLC die area and the HBF stack size. No headline runs until confirmed.
- **(d) cost metrics added** (hit rate vs (b); HBF bytes per reuse-hit block). At the design point, (d)'s gain is mostly not lost reuse: Conv reuse −2.5%, T&A within its range.
- **CLAUDE.md capacity section rewritten** for writing ranks (on disk; gitignored). Pending LaTeX sync gains the citation and unit items.

2026-10-11 — PHASE 14, STEP 3a controls (pool sized for 4 writing ranks)

- **Option (B):** every tier is sized for the 4 TP-0 writing ranks. PCM 71,104 blocks is the 43.4 GiB/GPU design point; HBF is 5,033,164 blocks. The TP-1 stream is a mirror and is not simulated. `store_partition_ranks` must equal the number of writing ranks (= DP size). Tests **237**.
- **Capacity effect (71,104 vs 142,213):**
  - (d) HBF **−71% / −74%**, ranges do not overlap.
  - (b)/(c) HBF +1.2% to +4.6%.
  - (b)→(c) 50.60% / 50.97%; (b)→(d) **99.64–99.68% / 99.66–99.72%**.
  - PCM residency halves.
- **Sharing effect (per_rank vs global at 71,104):** (d) HBF −13.8% / −24.5%, ranges do not overlap. (b)/(c) ≤ 0.22%.
- **Audit:** every "43.4 GiB/GPU" attached to 142,213 means 86.8 GiB per writing GPU. The PHASE 10 knee is ~38×, not 19×. Requests-to-wear-out are 2× too high. B/req are per TP shard. Batch is restated per active scheduler.
- The headline is **not** replaced; decision pending.

2026-10-11 — PHASE 14, STEP 3a (cross-rank eviction)

- **New `store_partition` (`global` | `count` | `per_rank`) and `store_partition_ranks`.** Default byte-identical; `count` is behaviourally stock; tests 221 → **236**; code `cd62320`.
- **Cross-rank evictions are 73.6–75.9% of all memory-tier evictions** in every config, at the 75% expected for 4 equally loaded ranks under a rank-blind LRU.
- **Found: only 4 of 8 ranks write to the store** (TP-0 only; see the constraint above). So `per_rank` at 8 partitions left half the PCM pool unused (71,104 usable blocks).
- **Under `per_rank`:**
  - (b) HBF +1.4% / +1.9%; (c) +2.1% / +4.5%.

  - (d) HBF **−75% / −76%**, outside the pooled range, so an effect by the rule. But it is confounded with the capacity halving.

- **Open:** how the TP-0-only model maps to hardware. That decides both the per_rank comparison and the PCM capacity bookkeeping.

2026-10-11 — PHASE 14, STEP 3 follow-up (pooled bound)

- **Pooled cap-48 (d) bound,** N = 16 per trace: stock plus burst replicates at three j. Worst-case reduction **98.72% / 98.44%**. (d) HBF B/req 32.0–43.0 M / 17.2–26.3 M. Ratios ×78.3–×105.2 / ×64.3–×98.8.
- **Over every cap tested** (46 runs per trace) the worst case is 98.66% / 98.28%.
- **Candidate:** outcomes cluster on a few discrete values.
- **Withdrawn:**
  - the periodic-burst-train candidate (an aperiodic train reproduces the stock outcome);
  - T&A's j-dependence call (its spread is non-monotone in j).
- **The T&A cap 48→256 +26.86% at j = 0.05** is part of a non-monotone swing, not a batch effect.

2026-10-11 — PHASE 14, STEP 3 (concurrency sweep for (d))

- **Sweep:** (d) at caps 8–256, both traces, 3 seeds per cap, burst jitter at j = 0.05 × gap. That j is the stricter test; at 0.45 the threshold would be 26–30%. 48 runs. The cap-48 runs reproduce the prelude runs byte for byte. (b) reuses the PHASE 9 STEP 4 rows, plus new stock T&A cap-192 and cap-256 runs, which PHASE 9 never had.
- **Verdict: "not shown" on both traces.** The means are non-monotone. The cap 8→256 change is +15.01% / +2.45%, against largest spreads of 23.76% / 20.22%.
- **Flagged:** T&A (d) means vary 40.5% non-monotonically, with 18/28 cap pairs non-overlapping. So (d) is not invariant either.
- **Batch gaps:** (d) vs (b) grows with cap, to −4.08% (Conv) and −6.60% (T&A).
- **The (b)→(d) reduction holds at every cap:** 98.69–98.99% / 98.32–98.88%.
- **Extra check at j = 0.005:**
  - T&A seed 4 and Conv seed 5 reproduce the stock store outcome exactly.
  - Outcomes repeat in discrete values.
  - T&A spread against j is non-monotone (33% → 14% → 30%).
  - The "periodic burst train" explanation is a candidate only: an aperiodic train also reproduces the stock outcome.
- Notes after the prelude review: the request-mode wording is revised, the reference of every delta is stated, and T&A's j-dependence is marked borderline.

2026-10-11 — PHASE 14, STEP 3 prelude

- **Jitter size.** (d) cap 48 at j = 0.05 × gap (15.685 / 30.287 ms) spreads **8.55% / 14.06%**, against 25.99% / 30.01% at j = 0.45. That is a ratio of 0.33 / 0.47, below half on both traces. **The spread depends on j**, so (d) ranges now carry their j. It is still above 5% at small j.
- **(c) replicates.** HBF spread 0.20% / 0.25%. The same-seed (b)→(c) reduction is **51.07–51.25% / 52.02–52.25%**, which contains the headline 51.10% / 52.18%.
- **Request-mode table note.** Its (d) HBF shifts are within (d)'s spread, or at its edge (T&A cap 256: −18.88% against 19.26%), so they are not readable as effects. Counter-evidence recorded: all four shifts are negative, and 12/20 runs fall outside the clean range.

2026-10-07 — PHASE 14, STEP 2b revised (clean replicates)

- **New `--arrival_jitter_mode`.** `request` is the earlier per-request jitter, now named and treated as a workload variant. `burst` gives one offset per burst on the replay clock and **fails the run unless arrival order equals file order**. Byte-identical at default; tests 213 → **221**; code `20695c3`.
- **Burst gaps in replay time are almost constant:** 313.7 ms (conv) and 605.7 ms (T&A). That gives j = 0.45 × min = **141.168 / 272.582 ms**, a methodology choice. The order guard never fired in 40 runs.
- **Request-mode runs recorded as an intra-burst-order sensitivity result.** 11–13% (conv) and 27–29% (T&A) of continuations end up before their predecessor. (b) hit rate −2.8% to −3.4%.
- **Clean replicates:**
  - (b) is neutral against stock: HBF ±0.15%, hit rate within +0.52%.
  - (d) HBF spread is **26% / 30%** at cap 48 and 16% / 19% at cap 256.
  - The stock T&A (d) run is **below all five** replicates at cap 48 and **above all five** at cap 256.
- **Q1:** conv yes (the replicates cover the STEP 2 range). T&A no: the STEP 2 low end is the stock run itself, and its high end is 2.4% above the replicate max.
- **Q2:** the single-run +70.10% exceeds both spreads but is not reproduced. Replicate means differ by +17.56% and the ranges overlap, so batch is not shown to drive it.
- **STEP 3 rule:** the spread is > 5%, so STEP 3 runs 3 replicates per point.

2026-10-07 — PHASE 14, STEP 2b (replicates)

- **`--random_seed` is inert.** (d) T&A seeds 0/1/2 are byte-identical. The seed reaches only synthetic workloads and MoE routing.
- **New flags `--arrival_jitter_ms` / `--jitter_seed`.** Default 0, and stock output stays byte-identical. Tests 205 → **213**. Code `5108029`.
- **The 1 ms resolution premise is false.**
  - The trace's timestamps are whole seconds, clustered ~3 s apart: 532 / 276 bursts in W = 5,000.
  - ±0.5 ms jitter only permutes order within a burst, and 24.7% / 56.5% of continuations share a burst with their predecessor.
  - Result: (b) hit rate is lower in 20/20 jittered runs and HBF B/req higher in 20/20. The replicates are a workload variant, not noise around the stock run.
- **Q1:** replicates alone do not cover the STEP 2 (d) range on either trace. The widths are comparable (1.79× / 0.89×); the location is shifted.
- **Q2:** (d) T&A cap-256 vs cap-48 replicate means differ by +16.53%, and the ranges overlap. The single-run +70.10% lies inside the pairwise range (−20.5% to +70.7%). Batch is not shown to drive it.
- Open: a perturbation that keeps within-burst order. Awaiting a decision.

2026-10-07 — PHASE 14, STEP 2

- New flags, both default-stock:
  - `store_timing_mode` (`charged` | `free`).
  - `count_aligned_save_skips`, instrumentation only.
- Stock output is byte-identical to `ced3806` on (b) and (d), both traces. Tests: 190 → **205**. Code commit `9e8b4d1`.

- **The pre-registered 5% rule failed, on (d)'s HBF stream only.**
  - (b) HBF ≤ 0.18%, and every memory-tier/PCM stream ≤ 0.54%: not contaminated by timing.
  - (d) HBF: Conv −1.0% to −10.9%; T&A +4.3% to +57.0%.
  - The response is non-monotone: PCM write latency 0.25× and 4× both raise T&A (d) HBF (+41.5%, +34.4%).
- The PHASE 9 cap-48 (d) value sits at an edge of the spread on both traces, so ×98.8 (T&A) is the top of ×62.9–×98.8. The reduction stays 98.41–98.99%.
- Not run: seed/jitter replicates. They would separate perturbation sensitivity from timing.

2026-10-07 — PHASE 14, STEPs 0–1

- Branch `phase14-fixes` cut from `main` at `ced3806`.
- **STEP 0 gate passed.** Configs (b) and (d), cap 48, both traces: all 20 values in the "Matched batch (cap 48)" table reproduce to the last digit. The result files are also byte-identical (stripped) to the PHASE 13 runs.
- **STEP 1 is paperwork only:**
  - Recorded as an unquantified limitation: pool keys embed the tp/pp/dp rank, yet one LRU and one capacity counter serve all 8 workers.
  - Flagged "~10^8 PCM endurance" (findings.md ~1431, ~1892) as unsourced. PCM endurance stays OPEN.
  - Recorded as assumed: the `pcm_*` timing defaults, and the HBF write timing (200 µs, 3.0 GiB/s), which is TokenSim's stock SSD default.
- **This report's headline now uses the PHASE 9 STEP 3 cap-48 figures:** 51.10/52.18%, 98.76/98.99%, ×80.4/×98.8. These replace the cap-256 reductions and the PHASE 8 multipliers ×90.31/×86.39. Every lifetime ratio is now paired with bytes/request.
- The cap-256 table was labelled "cap 256 / 96". All its rows are cap 256, so the label is corrected.

2026-10-01 — PHASE 13 and paper draft

- **PHASE 13** appended to `findings.md`: a correction, a limitation, a trace option, and sensitivity runs. No simulator source changed.
- **Correction to PHASE 12.** "The previous output is a minority of what a continuation adds" was wrong for Conversation and the arithmetic was inverted: median `delta/out` = 1.486 makes the output **67.3%** of added tokens, a *majority*. Tool&Agent is **15.7%**, a minority. Appended, not edited in place.
- **Tail-divergence limitation recorded.** The traces hash at 512 tokens while the simulator blocks at 16, so a continuation cannot reuse its predecessor's prompt tail in **97.9% / 98.7%** of pairs — median 22 / 14 blocks, **1.35% / 1.95%** of distinct blocks. All reported hit rates are therefore **floors**. Within-workload comparisons are unaffected.
- **`build_trace.py --share_partial_tail`** added, default off and byte-identical when off. **It does not work as a fix:** on config (b) it *lowers* the hit rate by 26.9% / 6.6% and *raises* HBF bytes/request by 7.4% / 6.9%, because the rule picks each continuation's predecessor independently and siblings that previously shared a chain diverge earlier. 70.8% of Conversation's chosen predecessors share only one 512-token chunk.
- **Two numbers shown to be far more workload-sensitive than reported:** the `if_read` hit-rate cost (−40.7% → −27.7%, −9.6% → −3.3%) and the HBF lifetime *ratio* (80.4× → 230.3×, 98.8× → 213.0×). **Bytes per request is the robust figure; the ratio is not.**

- All four original-trace runs reproduce PHASE 9 STEP 3 exactly, **config (d) included** — not previously re-measured.
- **MICRO paper draft** written to `../Latex/pcm-kv-micro.tex`. Not compiled — no LaTeX toolchain on this machine.
- Tests: 177 → **190**.

2026-09-28 — PHASE 12, the falsifier

- Pre-registered falsification test of the mechanism: if HBM eviction drove far-tier writes, HBF bytes/request would track HBM occupancy. Wired HBM evictions into the store behind `hbm_evict_save_policy`. Result **+0.00% / −0.07%** across an 11× batch range — **hypothesis dead**.
- Verdict covers **prompt-block eviction only**; no decode block can reach the hook (`hbm_evict_unkeyed_blocks = 0` in every run).
- Mechanism measured: eviction *volume* barely moves with batch (+1.66% / +1.42%), and **97.4–98.0%** of evicted keys are already in the store, so their put is a zero-byte LRU refresh.
- Fixed a silent export defect — `LLMResult` is a pydantic dataclass and was dropping undeclared stats keys. Audited: **no pre-PHASE-12 counter was ever affected**.
- Workload and harness rebuilt from scratch (nothing survived on the machine); reproduces **twelve** independently recorded PHASE 9 values exactly, first try.
- Tests: 158 → 177.

Superseded numbers (before the 2026-10-12 headline; PCM 142,213 / HBF 10,066,329 blocks, W = 5,000)

> Config (b) = a write-through DRAM buffer (same pool size as the PCM tier) in front of HBF, write-through + always — not "HBF only". (c) = +PCM, write-back + always. (d) = +PCM, write-back + if_read. All at `H3-B200-KVONLY`, LLaMa2-70B-GQA TP2/DP4, FP16, block 2,621,440 B, PCM pool 142,213 blk (347.2 GiB), HBF pool 10,066,329 blk (24 TiB), W = 5,000, 30 QPS.

HBM-saturated (cap 256, both traces), PHASE 9 STEP 3 (revised) — saturated, NOT the headline

> **Saturated operating point.** Not matched to the cap-48 standard. Batch gaps are larger here: **(d) vs (b) −4.72% (Conversation), −7.68% (Tool&Agent)**, and (c) vs (b) −1.63% / −2.31%. Use the cap-48 table below for headline figures.
>
> **Label correction (2026-10-07).** This heading used to say "cap 256 / 96". All six rows are from `--max_parallem_sum 256` (findings.md ~line 3510). The T&A rows match the cap-256 runs, e.g. (b) 1,695,909,675 B/req, and not the cap-96 STEP 4 point (1,678,879,752 B/req).

	(b)	(c)	(d)
Conversation HBF write blocks	6,461,439	3,150,432	**80,968**
HBF bytes/request	3,387,654,930	1,651,733,692	**42,450,551**
prefix hit rate	0.1631	0.1641	0.0942
achieved batch, raw `batch_mean_per_scheduler` (÷ 8 workers)	43.61	42.90	41.55
batch per active scheduler (TP group) = 2 × raw	**87.22**	**85.80**	**83.10**
reduction vs (b)	—	**51.24%**	**98.75%**

	(b)	(c)	(d)
Tool&Agent HBF write blocks	3,234,691	1,534,945	**55,703**
HBF bytes/request	1,695,909,675	804,753,244	**29,204,414**
prefix hit rate	0.4014	0.4033	0.3733
achieved batch, raw `batch_mean_per_scheduler` (÷ 8 workers)	48.44	47.32	44.72
batch per active scheduler (TP group) = 2 × raw	**96.88**	**94.64**	**89.44**
reduction vs (b)	—	**52.55%**	**98.28%**

Matched batch (cap 48), PHASE 9 STEP 3 — the reproduction gate

	HBF write bytes	HBF blocks	hit rate	reuse hit blk	raw `batch_mean_per_scheduler` (÷ 8 workers; the gate value)	batch per active scheduler (TP group)
(b) Conversation	16,853,376,696,320	6,429,053	0.1643	670,585	23.167889	**46.335778**
(d) Conversation	209,508,106,240	79,921	0.0974	397,596	22.378200	**44.756400**
(b) Tool&Agent	8,483,936,665,600	3,236,365	0.4037	1,174,042	22.885048	**45.770096**
(d) Tool&Agent	85,844,295,680	32,747	0.3650	1,061,727	22.429517	**44.859034**

Any new campaign must reproduce this table to every digit before its own numbers are trusted.

> **Units, from PHASE 14 STEP 3a:**
>
> - **Batch.** Only the 4 TP-0 workers are scheduled. Raw `batch_mean_per_scheduler` divides by all 8 workers, so batch is now quoted **per active scheduler (TP group) = 2 × raw**. The raw column stays because it is the reproduction-gate value.
> - **Bytes.** All B/req and write-byte figures are **per TP shard** (one of 2).
> - **Capacity.** 142,213 blocks is **86.8 GiB per writing GPU**, 2× the design.

43.4 GiB/GPU design point — CONTROL, not the headline (PHASE 14 STEP 3a)

PCM 71,104 blocks (4 writing ranks × 17,776) and HBF 5,033,164 blocks, cap 48. (b) and (c) are single runs; (d) is a range over 4 runs (unjittered + burst j = 0.05 × gap, seeds 1–3). B/req per TP shard. **The headline above (142,213) is unchanged pending a decision.**

	(b) HBF B/req	(c) HBF B/req	(d) HBF B/req	(b)→(c)	(b)→(d)	(d) hit rate	batch / active sched
Conversation	3,411,597,591	1,685,218,918	10.9–12.2 M	50.60%	99.64–99.68%	0.0894–0.0946	44.9–46.3
Tool&Agent	1,730,365,358	848,324,723	4.85–5.94 M	50.97%	99.66–99.72%	0.3663–0.3666	45.0–46.1

Against 142,213, the capacity effect on (d) HBF is −71% / −74% (ranges do not overlap). On (b) and (c) it is +1.2% to +4.6%. Per-rank partitioning at the same capacity moves (d) HBF a further −13.8% / −24.5% (ranges do not overlap), and (b)/(c) by ≤ 0.22%.

Workload footprint

	prompt-block instances	distinct blocks	bytes	dedup
Conversation	4,080,885	2,695,220	6,580 GiB	1.51×
Tool&Agent	2,908,548	1,397,479	3,412 GiB	2.08×

Pending LaTeX sync

> **2026-10-22: the single LaTeX rewrite applied every item below** (chunks 1–4, each reviewed by the author). Items are kept as a record and ticked.

Still open for the paper:

- The file has **never been compiled**: there is no LaTeX toolchain on this machine. A brace, environment, reference and citation check passes.
- Bibliography complete and verified (2026-10-22): `h3` (Ha, Kim, Kim, IEEE CAL 25(1) 49–52), `tokensim` (Wu *et al.*, arXiv:2503.08415), `mooncake` (Qin *et al.*, FAST'25 155–170; the FAST title differs from the arXiv title), `vllm` and `mqsim` (pages added).
- The only figure is the endurance sweep (pgfplots).
- The venue template (IEEEtran vs acmart) needs checking against the current call for papers.

One LaTeX rewrite, after STEP 4. Every item below is listed, not applied.

- ☑ **DRAM density dropped (2026-10-22).** The paper makes no DRAM-density comparison. Durability leads, and the placement claim ("inside the existing HBF stack footprint") is the only area statement.
- ☑ **"Why not DRAM" (2026-10-17).**
  - Lead with **durability**: under write-back, acknowledged data lives only in the staging tier, so a volatile buffer would lose it on power failure. Include the recomputable-KV caveat.
  - Area/density is supporting evidence only.
  - **Do not cite the HBM3/HBM3E newsletter figures.** They are placeholders (NEEDS PRIMARY SOURCE).
    - Until an ISSCC/IEDM HBM3E die-area paper is sourced, give no DRAM density number in the paper. At most write "per-die density is higher than HBM's, subject to TSV overhead".
    - Never use the ~0.33 Gb/mm² TSV-penalised figure (author's illustrative arithmetic).
- ☑ **FRAMING (2026-10-17). This supersedes "(c) as the primary result" in the NEW HEADLINE item below.** Structure the abstract, contributions, `sec:headline` and the Conclusion as three separately labelled claims.
  1. **Measured, an identity (baseline characterisation, not a contribution).**
     - Write-through costs one extra HBF write per admitted block. Removing it halves HBF writes whatever the HBF capacity.
     - 8 runs: raw 50.22–50.25%, residue-adjusted 49.26–49.29%, finite vs unlimited ≤ 0.01 pt.
     - HBF evictions are immaterial (≤ 0.09% of HBF bytes, ≤ +0.11% hit rate).
     - Phrase it as "we quantify a structural cost in the baseline and confirm it is capacity-independent". It is **not** a PCM-media effect, and not a surprising finding.
  2. **Argued, not measured — say explicitly that TokenSim does not support it.**
     - Durability under write-back, with the caveat that KV is recomputable, so the loss is recomputation rather than correctness.
     - In-package capacity per area. The DRAM comparison figure is OPEN.
     - Add the sentence: "*A same-capacity volatile buffer would write identical HBF bytes; the case for PCM rests on durability and in-package density, not write volume.*"
  3. **Measured, the contribution: (d) read-gated demotion.**
     - It drops 98.7–99.0% of (c)'s HBF writes.
     - It keeps 53–54% / 88% of (b)'s reuse hits, **against 48% / 86% for no store at all**, i.e. 9–15% of the store's reuse benefit. Always pair the retention figure with the no-store figure.

        - PCM serves 71–77% of (d)'s store-served tokens.
        - Re-admission after an `if_read` drop is a share of PCM writes, N = 4 (see report.md).
    - Retire every "(c) is the primary result" and "PCM halves flash writes" phrasing.
- ☑ **NEW HEADLINE (2026-10-12).** Spill-only + vLLM-style lookup + honest read bookkeeping, 43.4 GiB/GPU per_rank, cap 48.
    - Rewrite the abstract, contributions, `tab:headline`, `sec:headline` and the Conclusion around **(c) as the primary result** (~50.5–51.3% in every model, no hit-rate cost).
    - Present (d) as the trade-off end: worst-case 99.36% / 99.12%; hit rate −37.5 to −39.1% / −6.8 to −7.7%; keeps 15–19% / 10–21% of the store's reuse benefit.
    - Add the no-store control, the PCM-served share, the residue bias, and the copy-at-admission variant ((d) ≡ no store).
    - **State that the headline was chosen on modelling grounds and lowers (d)'s reduction.** STEP 4 full-trace numbers replace the W = 5,000 ones if they differ.
- ☑ **STEP 4 (full traces) — DECIDED: full traces are the headline window** (2026-10-12). W = 5,000 can appear as an earlier reference only.
    - (c) ~49–50%: raw 50.23 / 50.25%, residue-adjusted 49.29 / 49.26%.
    - (d) worst case 99.45 / 99.33%; (d) hit rate −47% / −12% vs (b); store benefit kept 9–15%.
    - Note the HBF-capacity regime on full traces: (b)/(c) evict, (d) does not.
- ☑ **Describe (b) exactly.** It is a 71,104-block write-through DRAM buffer in front of HBF, serving 6.9% / 7.2% of (b)'s store-served tokens on full traces. Not "HBF only": retire that label wherever it appears.
- ☑ **(c)'s halving is a write-policy identity** ((b) HBF = admissions + demotions, (c) HBF = demotions). State the HBF capacity check result. Framing decided 2026-10-17: see the FRAMING item above.
- ☑ **Read path:** PCM or HBF serve directly into HBM. HBF hits are not re-admitted to PCM (`store.py:179-182, :234-237`).
- ☑ **Architecture / method:** describe spill-only entry (HBM evicts → PCM), the vLLM-style lookup beyond the HBM hit, and read credit on actual serve. Retire "each prompt block is copied to PCM at admission" as the model description; it is the variant.
- ☑ **Retract** any read-back mechanism text ("one read in PCM passes `if_read`"), and any (d) variability, concurrency or j-size discussion presented as a property of the design.
- ☑ **Architecture/Methodology wording (PHASE 14 STEP 3b).** State plainly that **"each prompt block is copied to PCM at admission"** (the prefill save). Nothing should imply that HBM evictions spill into the store. Lines that currently imply it: l.102 ("a KV store that absorbs spill from HBM"), l.170 ("HBF receives the spill stream from a KV store") and l.379 ("The residual is the genuine spill"). Reword them to "the store's overflow/demotion stream".
- ☑ **From-zero result, one sentence:** "*If blocks enter the store only on HBM eviction and the store is looked up from the first prompt block, the HBM-resident prompt head is never stored and the store serves almost no reuse (0 and 769 of 5,000 lookups); copy-at-admission is required for the store to be useful under that lookup.*"
- ☑ **Robustness paragraph for Evaluation, rewritten from set B** (replaces the draft below). Draft: "*Robustness to the PCM entry model. We also let blocks enter the store only when HBM evicts them, with the store queried, as in vLLM's connector interface, only for tokens beyond those already in HBM. PCM then behaves as a victim cache and serves real reads. Write-back halving is unchanged (50.5% / 51.3%), and read-gated demotion still cuts flash writes by at*

least 98.96% against the flash-only baseline. But (d)'s residual flash traffic is 1.6–4.1× the copy-at-admission figure (18.8–24.4 MB vs 6.0–11.7 MB per request, conversational). By our pre-registered rules the headline numbers therefore depend on how blocks enter PCM."

- ☑ **Read-gate accounting (PHASE 14 correction).** Any (d) mechanism text must say that under copy-at-admission PCM serves no reads, and that the gate was opened by lookup touches (`store.py:125`). Wait for the headline-lookup decision before rewriting.
- ☑ *(superseded by the set-B paragraph above)* **Robustness paragraph for Evaluation (PHASE 14 STEP 3b)** — one short paragraph; headline tables unchanged. Draft: "*Robustness to the PCM entry model. If blocks enter the store only when HBM evicts them, rather than at admission, the result does not carry over. The HBM-resident prompt head is then never stored, and prefix-ordered lookups stop at it: the store serves 0 of 5,000 lookups (conversational) and 769 of 5,000 (tool/agent). Read-gated demotion therefore never passes, and (d) writes essentially nothing to flash (0 and 0.37 MB per request). By our pre-registered rules the headline does not hold under spill-only entry (rule iii fails on both traces, rule ii on tool/agent), so the result depends on how blocks enter PCM.*"
- ☑ **0.62 Gb/mm² cited (2026-10-11).** `\cite{ehrmann2022pcm}` is at the contribution item and the density line, replacing "derived from published die parameters". The bib entry is complete (supplied fields).
- ☑ **Mechanism paragraph for (d).** Within the window, ~98% (T&A) / ~70–76% (Conv) of the HBF writes (d) avoids at the design pool, against 2× capacity, are blocks that pass `if_read` at the larger pool but are not read back from HBF. Read-back figures are in-window lower bounds.
- ☑ **Still uncited: 35 mm² (PCM die) and 29 Gb/mm² (TLC density).** The 29 Gb/mm² unit stays OPEN until sourced. Note the 35 vs 35.3 mm² rounding (< 1% capacity).
- ☑ **Add a limitation line:** 2nd-gen (4-deck) XPoint roughly doubles density, which is approximately the 86.8 GiB/GPU point (142,213, global pool). No runs for it.
- ☑ **Wording rule:** nowhere call 0.62 Gb/mm² "conservative" overall. A smaller pool lowers (c)'s reduction and raises (d)'s.
- ☑ *(superseded by the items above)* **Uncited die figures (PHASE 14 decision record).** l.207–216 give 35 mm² per PCM die, 0.62 Gb/mm² PCM density and 29 Gb/mm² TLC density with **no `\cite`**.
  - **TLC sets two things in this paper:** PCM capacity (the PCM die is the area of a 1 Tbit TLC die) and HBF endurance (3000 P/E). Both need a source.
  - **Unit.** l.210–212 turn 43.4 Gbit/stack into 5.43 GiB, i.e. binary Gbit. If the density source is decimal, PCM is 40.42 GiB/GPU. The "1 Tbit die ≈ 35.3 mm², 128 GiB" (l.215–216) also assumes binary Tbit; decimal gives 34.5 mm². Fix all three together once confirmed.
- ☑ **Headline configuration change (PHASE 14 decision record) — numbers now available (headline set).** The headline becomes design capacity + `per_rank`: - (b)→(c) 50.81% / 50.99%; - (b)→(d) worst case 99.66% / 99.63% (N = 16), with (d) HBF B/req 5.95–11.70 M / 4.08–6.35 M; - (d) hit rate vs (b) −43.0 to −45.9% / −8.6 to −9.2%; - HBF bytes per reuse-hit block and HBF read-back share as in findings.

> Earlier wording: Every
> `tab:headline` number, the abstract and the Conclusion change. The
> current 142,213-block figures are the **86.8 GiB/GPU point**. Add the
> (d) cost metrics: hit rate vs (b), and HBF bytes per reuse-hit block.

- ☑ **8-writer assumptions (PHASE 14 STEP 3a audit).** List only, not fixed. Only 4 TP-0 workers are scheduled and write.
  - **Capacity.** "43.4 GiB per GPU" is attached to 142,213-block results: the abstract l.56 and l. 69, contributions l.112, l.288 (pool = 43.4 × 8), the Conclusion l.608. At 142,213 the pool is **86.8 GiB per writing GPU**. The design point is 71,104 blocks.
  - **Endurance budget.** l.172 gives `24 × 2^40 × 3000 = 7.92e16 B`, but only the TP-0 half (12 TiB) receives the counted per-shard bytes. The requests-to-wear-out figures (l.381–382, `tab:capacity` l.442–456) are **2× too high**. Their ratios are unaffected.
  - **Capacity sweep.** `tab:capacity` "GiB/GPU" column ×2. l.466: 5.3% of footprint is at 86.8 GiB per writing GPU; the design point is 2.6%. l.476: "822 GiB per GPU, 19× the budget" → ~1,645 GiB, ~38×. l.481–483: "1.8×–2.8×, 77–120 GiB" → 3.6×–5.6×, 154–240 GiB. l.71, l. 121 and l.613: "19×" → ~38×.
  - **Batch.** l.325, l.356, l.365 ("achieved batch/sched"), l.385, l.394 and `tab:batch` l.404 must be quoted **per active scheduler (TP group)**, which is 2× the old figures. Those were divided by 8 workers including 4 idle TP-1 workers. The cap is per active scheduler; cluster concurrency is 4 × cap.
  - **Platform.** l.278 "eight B200-class workers" and l.285 "single cluster-wide pool": add that the TP-1 stream is an exact mirror of TP-0 and is not simulated, and that bytes are per TP shard.
  - **Possible headline change.** If the 43.4 GiB/GPU design point (71,104 blocks) becomes the headline, `tab:headline` and every (d) figure change: (b)→(d) 99.64–99.72%, (b)→(c) 50.60% / 50.97%. Decision pending.

> What is in this report / `findings.md` but **not yet** in `../Latex/pcm-kv-micro.tex`. Clear entries as they are moved across.

- ☑ Bibliography is incomplete — five entries marked `% TODO` (author lists, volumes, page ranges, DOIs). **Must be filled before submission.**
- ☑ Paper has **not been compiled** — no LaTeX toolchain on this machine. `sudo apt install texlive-publishers texlive-latex-extra texlive-science`
- ☑ Confirm the MICRO template for the target year. The draft targets `IEEEtran[conference]`; the switch to `acmart/sigconf` is a commented one-liner in the header.
- ☑ Author block is a placeholder guess.
- ☑ No figures yet (next steps #6).
- ☑ PHASE 13's tail-sharing negative result is summarised in the Limitations section but not given its own subsection — decide whether it earns one.
- ☑ **Stale headline figures (PHASE 14 STEP 1d), listed only, not fixed.** Line numbers are in `pcm-kv-micro.tex` at `ced3806`:
  - Abstract, l.59–63: "51.2% / 52.6%", "98.8% / 98.3%" (cap-256, saturated) and "90.3× / 86.4×" (PHASE 8, batch-unmatched). Cap-48 values: 51.10% / 52.18%, 98.76% / 98.99%, ×80.4 / ×98.8, each paired with bytes/request.
  - `tab:headline`, l.340–370: the cap-256 saturated table is presented as the headline. It does not state the (d)-vs-(b) batch gaps (−4.72% / −7.68%).
  - Evaluation text, l.379–382: "98.75% and 98.28%" and "90.3× and 86.4×". No bytes/request is paired with the ratio.
  - l.384–385: "matched to within 4.7% and 7.7%" is the saturated spread. It should be labelled as such.
  - Conclusion, l.609–611: "51–53%", "98–99%", "90×".

- ☑ **Unsourced statements in the LaTeX (PHASE 14 STEP 1b/1c), listed only:**
  - l.177: PCM "write endurance is orders of magnitude higher". This is unsourced, and PCM endurance is OPEN.
  - l.273–274: flash writes "NAND-realistic at 200 µs and 3 GB/s". These are TokenSim's stock SSD defaults, not sourced HBF figures, and the knob is in GiB/s.
  - The PCM media timing (0.1/1.0 µs, 100/20 GiB/s, `config.py:81-84`) is never stated in the paper. It is assumed and should be disclosed as such.
- ☑ **HBF write bandwidth unit (PHASE 14 STEP 1).** The LaTeX (l.274, `\SI{3}{\giga\byte\per\second}`) and the project record (findings.md ~2802, ~3829, ~4531) say HBF writes run at "3 GB/s". The knob `ssd_write_bw_gbps = 3.0` is divided by 2^30 (`ssd.py:116-121`), so it is **3.0 GiB/s** (3.22 GB/s).
- ☑ **Headline ratio mismatch (PHASE 14 STEP 1e).** The prose says ×90.3 / ×86.4 (PHASE 8), but `tab:headline`'s own B/req give ×79.8 / ×58.1 (cap 256). The canonical figure is ×80.4 / ×98.8 (cap 48), quoted with its bytes/request.
- ☑ **(d) is now quoted as a pooled bound (PHASE 14 STEP 3).** This supersedes the item below. Use the worst-case reduction 98.72% (conv) / 98.44% (T&A), with N = 16 and the (d) HBF B/req range 32.0–43.0 M / 17.2–26.3 M at cap 48. Ratios only as ranges: ×78.3–×105.2 / ×64.3–×98.8.
- ☑ **(d) must be quoted as ranges (PHASE 14 STEP 2).** The abstract, `sec:headline` and the Conclusion must quote config (d) as a reduction range plus an HBF B/req range, not a single ratio: - reduction 98.76–98.89% (conv) / 98.41–98.99% (T&A) - (d) HBF B/req 37.3–41.9 M / 17.2–27.0 M - if a ratio is given at all: ×80.4–×90.3 / ×62.9–×98.8

  > Superseded by the STEP 2b clean replicates. Use reduction 98.76–99.05% / 98.44–98.83%, (d) HBF B/req 32.0–41.9 M / 19.8–26.3 M and, if a ratio is given, ×80.5–×105.2 / ×64.3–×85.7. State that the PHASE 9 T&A ×98.8 lies above every replicate.

- ☑ **Concurrency claim over-reaches (PHASE 14 STEP 1f, STEP 3).** Replace any "concurrency-invariant" wording for (d) in `sec:batch` and the Conclusion with: "*reduction ≥ 98.66% (conversational) / ≥ 98.28% (tool/agent) at every cap tested; residual HBF writes vary by up to 42% / 70% across caps and runs, with no trend*." - Those numbers are the worst case over all 46 (d) runs per trace at caps 8–256 (stock + burst-jitter replicates). The cap-48 pooled bound alone gives 98.72% / 98.44%, but caps other than 48 go lower. The T&A worst case is the stock cap-256 run. - Residual (d) HBF B/req spans 32.0–45.4 M (conv) and 17.2–29.2 M (T&A). - Keep "concurrency-invariant" for (b) and (c) only.